

Estructura básica de programación microcontroladores arduino/Atmel AVR

Estructura:

```
sketch_jul6a.ino
1 void setup() {
2   // put your setup code here, to run once:
3
4 }
5
6 void loop() {
7   // put your main code here, to run repeatedly:
8
9 }
```

El Sketch está dividido en 2 partes: setup y loop.

La función setup() se utiliza principalmente para definir la configuración del microcontrolador, mientras que la función loop() se utiliza para la definición del programa. La función setup() se ejecuta solo 1 vez mientras que la función loop() se está ejecutando constantemente.

Comentarios:

Los comentarios son líneas que se utilizan para añadir información y que el compilador ignora, permitiendo así añadir explicación sin usar memoria flash del micro, informar a otros programadores e incluso desactivar de forma temporal partes del programa para depuración.

Comentarios de una sola línea: //

Comentarios de bloque: /* */

```
1 void setup() {
2   // put your setup code here, to run once:
3
4 }
5
6 void loop() {
7   // put your main code here, to run repeatedly:
8   // esto es un comentario de línea
9   /* y este otro
10  | comentario es un comentario
11  | de bloque */
12 }
13
```

Directivas del preprocesador:

#define : se utiliza para dar nombres a valores constantes

#Include: se utiliza para incluir librerías externas

Sintaxis:

```
#include <libreriaExterna.h>
#include "libreriaInterna.h"
```

Punto y coma ';' se utiliza para finalizar una declaración.

Llaves {} se utilizan en varias estructuras para agrupar partes de código, se utilizan principalmente en: funciones, bucles, condicionales y en la estructura del código (loop y setup)

POO

Arduino utiliza una versión de C++ para microcontroladores, lo que implica que también se puede utilizar POO para programar los microcontroladores.

¿Qué es la Programación Orientada a Objetos?

La Programación Orientada a Objetos (POO) es una forma de organizar el código pensando en **objetos** en lugar de solo funciones.

Concepto simple: En el mundo real, todo son objetos:

- Un teléfono es un objeto
- Una televisión es un objeto
- Un coche es un objeto
- Una lámpara es un objeto

Cada objeto tiene:

- **Características** (datos): color, tamaño, estado
- **Acciones** (métodos): qué puede hacer

¿Qué es un Objeto en Arduino?

Un objeto es una "cosa" que tiene **datos** (propiedades) y **funciones** (métodos) asociadas.

Ejemplo: Un objeto Television

```
Objeto: Television
├── Propiedades (características):
│   ├── marcaTV = "Sony"
│   ├── canal = 5
│   ├── volumen = 20
│   └── encendida = true
└── Métodos (acciones):
    ├── encender()
    ├── apagar()
    ├── cambiarCanal()
    ├── subirVolumen()
    └── bajarVolumen()
```

¿Qué es un Método?

Un método es una **función que pertenece a un objeto**. Es una acción que el objeto puede realizar.

Diferencia:

- **Función normal:** sumar(5, 3) → función independiente
- **Método:** Serial.print("Hola") → función que pertenece al objeto Serial

La Notación Punto (.)

Para acceder a un método de un objeto, usas la **notación punto**:

```
objeto.metodo()  
objeto.propiedad
```

El punto (.) significa "**pertenece a**" o "**accede a**".

Ejemplo: Serial.print()

En `Serial.print("Hola")`:

```
Serial      .      print("Hola")  
|           |  
objeto      |      método con parámetro "Hola"  
           |  
           punto (acceso)
```

Traducción: "Usa el método print del objeto Serial para imprimir 'Hola'"

Más ejemplos de Serial:

```
Serial.begin(9600);      // Método: begin(). Pertenece a Serial  
Serial.print("Texto");   // Método: print(). Pertenece a Serial  
Serial.println("Texto"); // Método: println(). Pertenece a Serial  
Serial.read();           // Método: read(). Pertenece a Serial  
Serial.available();      // Método: available(). Pertenece a Serial  
Serial.readString();     // Método: readString(). Pertenece a Serial  
Serial.parseInt();       // Método: parseInt(). Pertenece a Serial
```

Todos estos métodos **pertenecen al objeto Serial** porque están relacionados con la comunicación serie.

Analogía: Control remoto

Piensa en el objeto **Serial** como un **control remoto de TV**:

```
// El control remoto (Serial) tiene botones (métodos)  
  
control.encender();           // Botón encender  
control.cambiarCanal(5);     // Botón cambiar canal  
int volumenActual = control.volumen; // Propiedad volumen
```

¿Quién crea los objetos?

Arduino proporciona objetos ya creados:

- Serial → para comunicación serie
- Wire → para comunicación I2C
- SPI → para comunicación SPI
- Etc.

Tú también puedes crear tus propios objetos (se verá más adelante).

Estructura general:

Arduino proporciona objetos → Tú usas sus métodos → Tu programa funciona

```
↓  
Serial.begin()  
Serial.print()  
Serial.read()  
Serial.readString()  
Etc.
```

Puntos clave a recordar

- 1º **Objeto:** Una "cosa" con características y acciones
- 2º **Método:** Una función que pertenece a un objeto
- 3º **Propiedad:** Un dato que pertenece a un objeto
- 4º **Notación punto (.):** Cómo accedes a métodos y propiedades
- 5º **Serial:** Un objeto que Arduino proporciona para comunicación serie
- 6º **Serial.method():** Significa "usar el método 'method' del objeto Serial"

Ejemplo completo que ya conoces:

```
void setup() {  
  // Serial es un objeto  
  // begin() es un método del objeto Serial  
  Serial.begin(9600);  
  
  // print() es otro método del objeto Serial  
  Serial.print("Hola");  
}  
  
void loop() {  
  // available() es otro método del objeto Serial  
  if (Serial.available() > 0) {  
    // read() es otro método del objeto Serial  
    char character = Serial.read();  
  }  
}
```

Todo son métodos del objeto Serial:

- Serial.begin() ← Método del objeto Serial
- Serial.print() ← Método del objeto Serial
- Serial.available() ← Método del objeto Serial
- Serial.read() ← Método del objeto Serial